BIF & Co.

SEMINAR NGINX

A super engine for your deployment needs

August, 2026

Start Slide

AGENDA & MEMBERS

1. Introduction & Capabilities
2. How it works & Configurations
3. Demonstration Setup & Video
4. Comparisons & Conclusion

23127020 • Biện Xuân An

23127043 • Hoàng Minh Giang

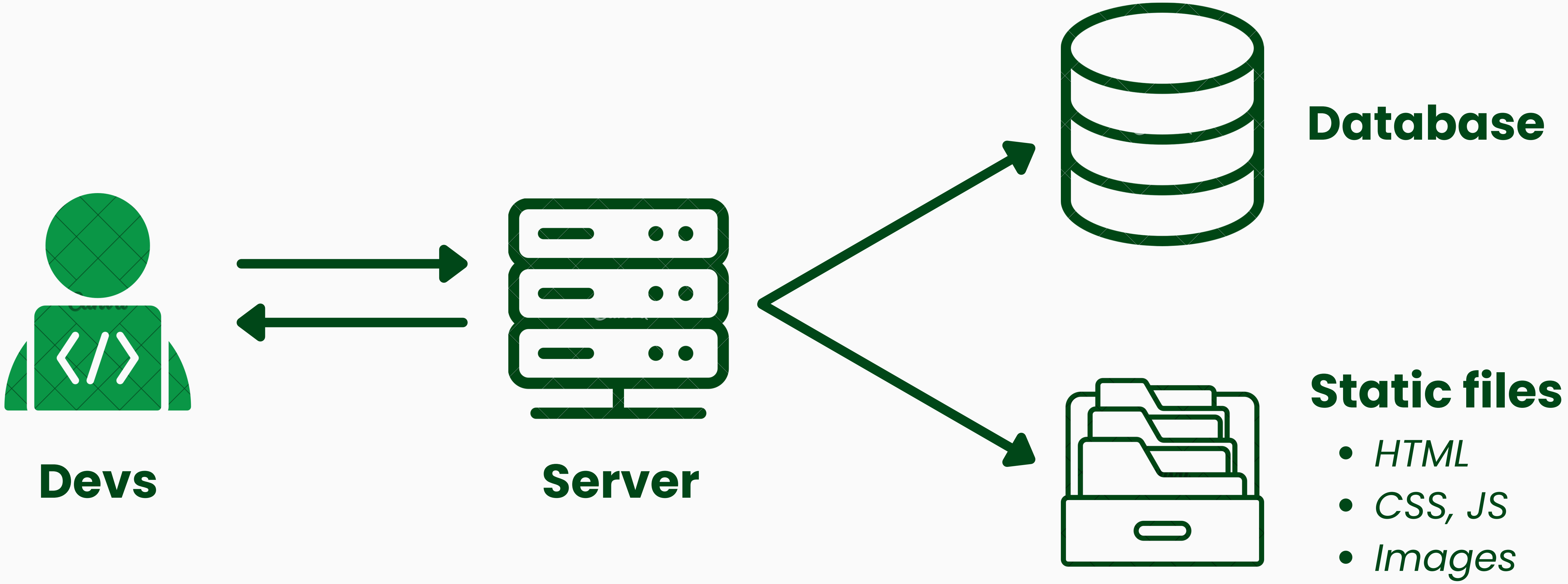
23127114 • Văn Ngọc Quý

23127136 • Lê Nguyễn Nhật Trường

INTRODUCTION & CAPABILITIES

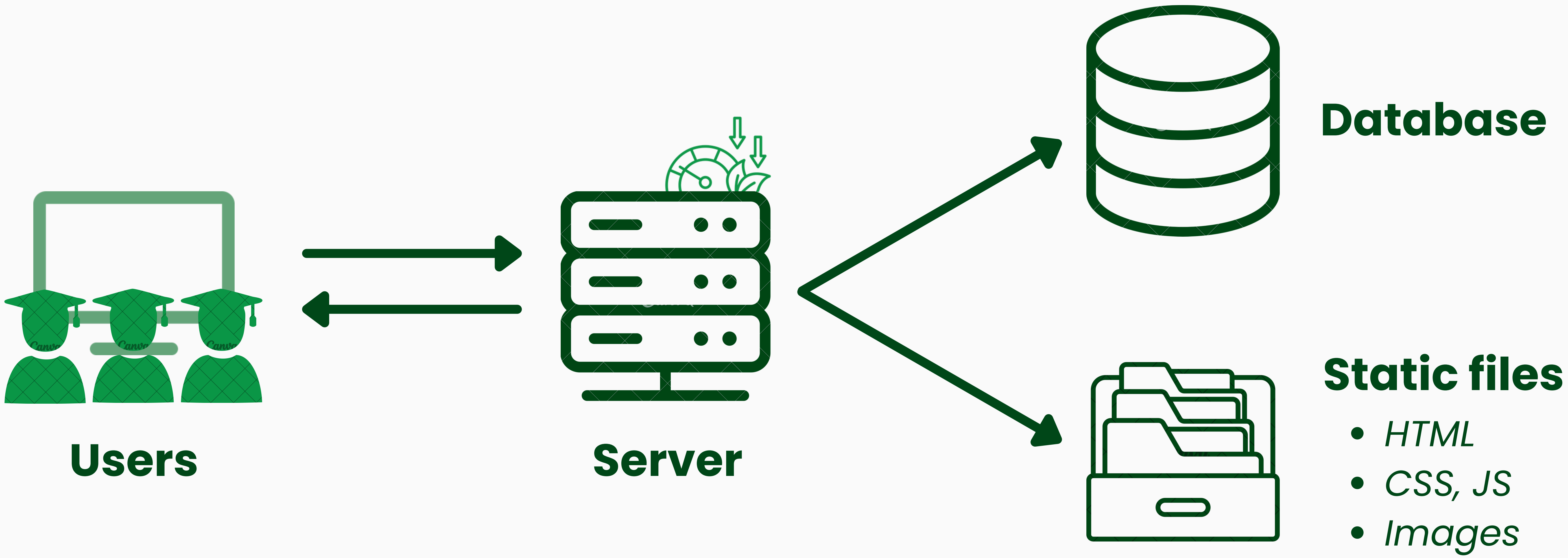
Speaker: Lê Nguyễn Nhật Trường

CASE STUDY: COURSE MANAGEMENT PORTAL



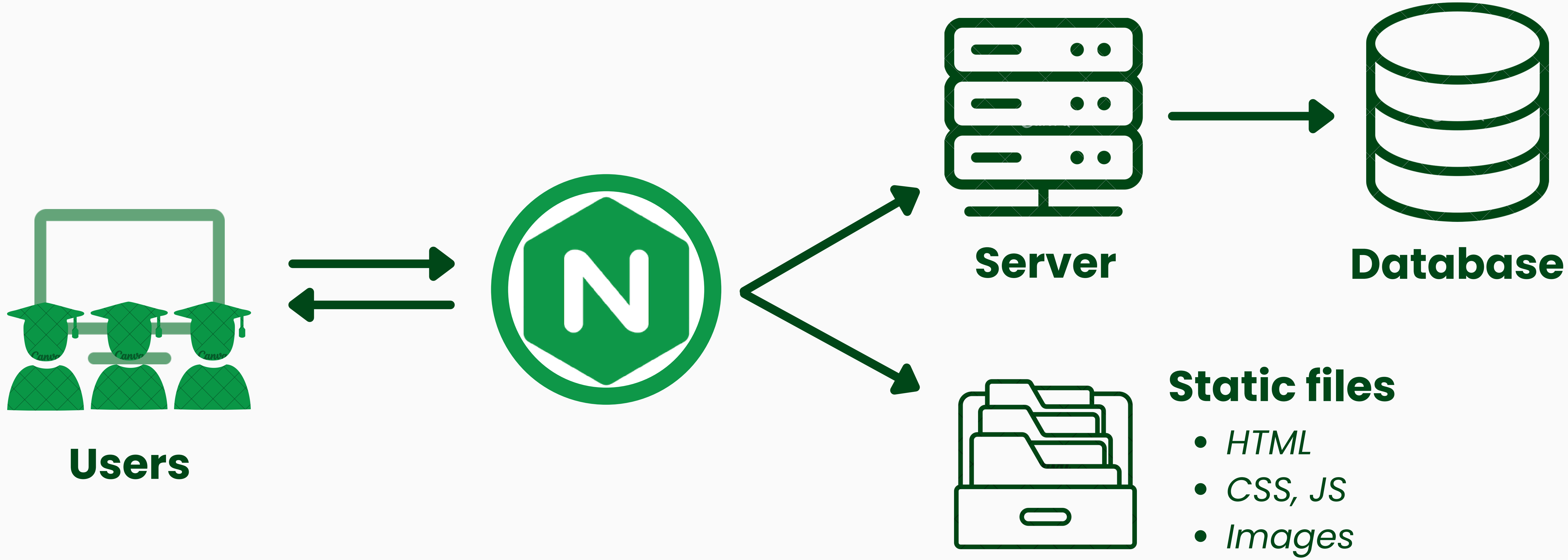
*From **localhost**: The development success*

CASE STUDY: COURSE MANAGEMENT PORTAL



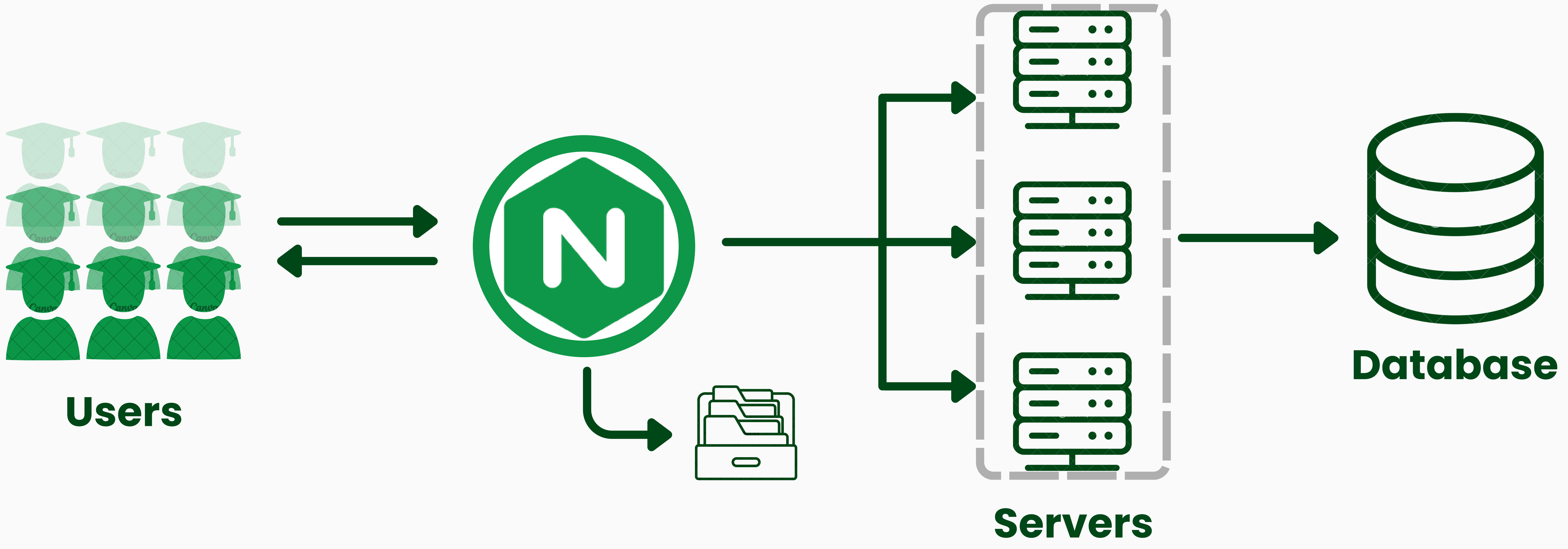
*To **production**: Too many problems!*

CASE STUDY: COURSE MANAGEMENT PORTAL



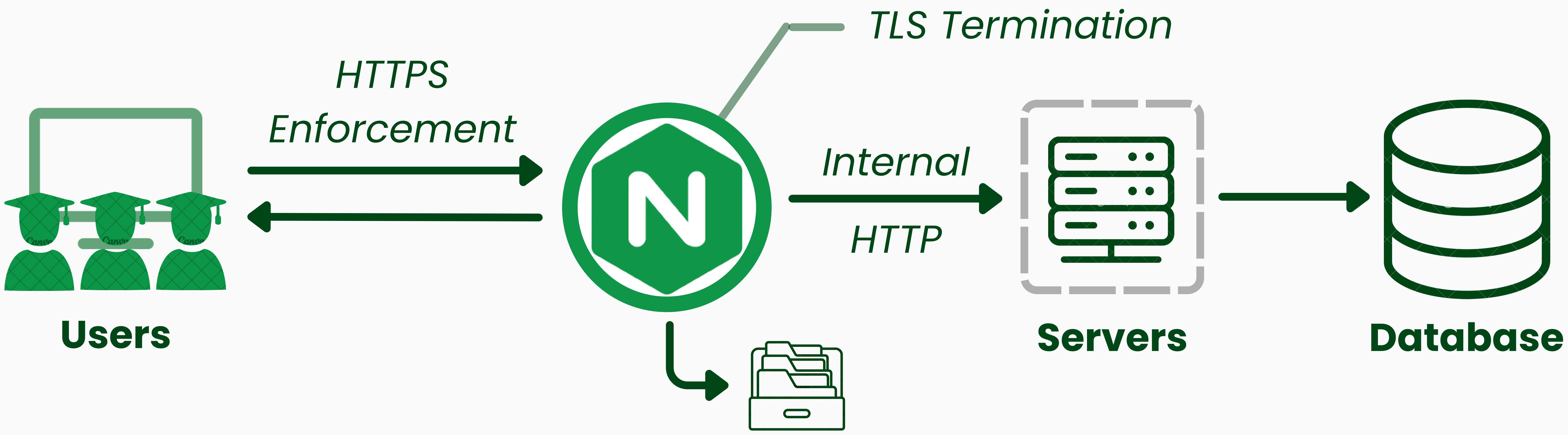
Nginx *jumps in: For* ***static files & reverse proxy***

CASE STUDY: COURSE MANAGEMENT PORTAL



Load balancing: *Easy horizontal scaling*

CASE STUDY: COURSE MANAGEMENT PORTAL



HTTPS / TLS handling: *Encrypt* ***sensitive*** *data*

THE NGINX STRENGTHS SO FAR

Web server & reverse proxy

Load balancing

HTTPS / TLS handling

But there are more...

THE MANY CAPABILITIES OF NGINX

There's so much more to uncover

Caching

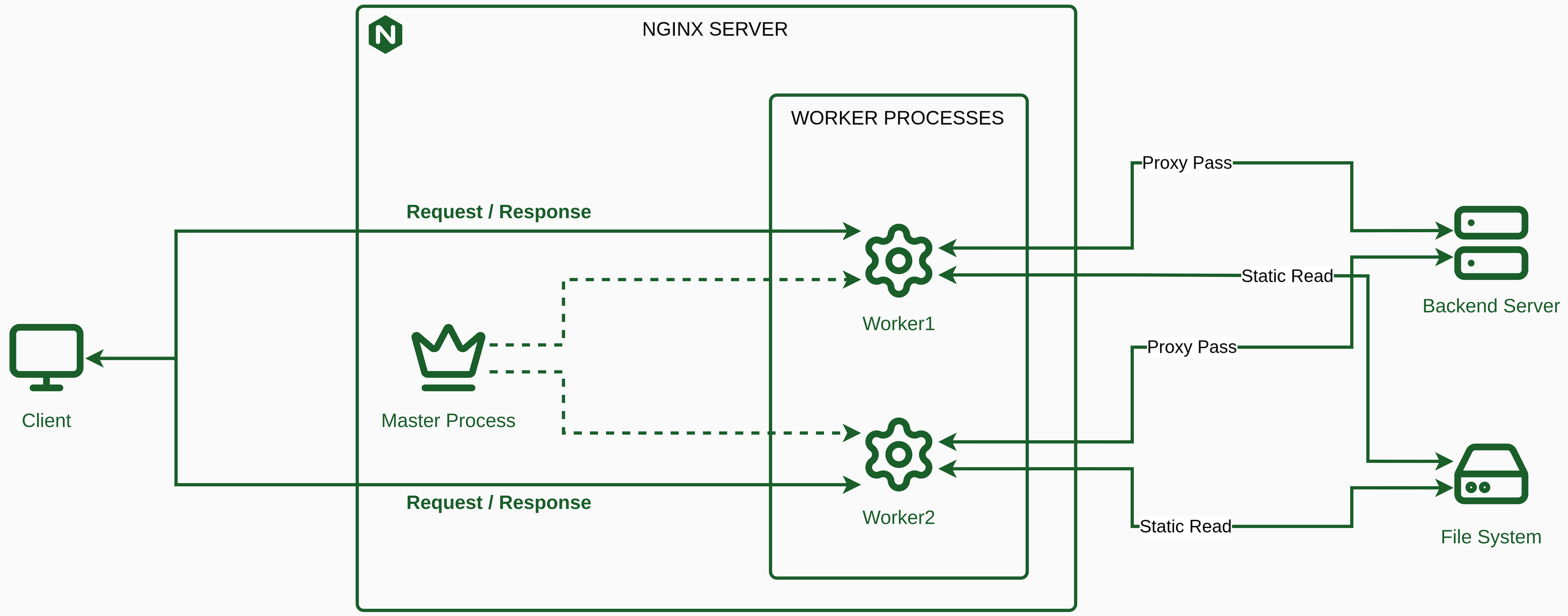
Path-based routing

Traffic control

Graceful reload

HOW NGINX WORKS & CONFIGURATIONS

Speaker: Văn Ngọc Quý


NGINX SERVER
WORKER PROCESSES
Request / Response
Request / Response
Client
Master Process
Worker1
Worker2
Proxy Pass
Static Read
Proxy Pass
Static Read
Backend Server
File System

NGINX INSTALLATION METHOD

Docker

docker run -p 80:80 nginx

Package Manager

sudo apt install nginx

Build from source

Global context: user, number of processes, master process location

Event context: Network connection limit

HTTP context: Web/proxy core settings, MIME types, logging,...

```
user www-data;
worker_processes auto;
pid /run/nginx.pid;

events {
    worker_connections 1024;
}

http {
    include /etc/nginx/mime.types;
    access_log /var/log/nginx/access.log;
    error_log  /var/log/nginx/error.log;

    # Include individual site config
    include /etc/nginx/conf.d/*.conf;
}
```

Server definition: port & domain name

Static files location

Reverse Proxy and Headers Location

```
server {
    listen 80;
    server_name api.myapp.com;

    # Serve static files directly
    location / {
        root /var/www/static/;
        index index.html;
    }

    # Reverse proxy to backend
    location /api/ {
        proxy_pass http://127.0.0.1:8080;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For
                         $proxy_add_x_forwarded_for;
    }
}
```

Load balancing

```
upstream backend_app {
    server 10.0.0.1:8001;
    server 10.0.0.2:8001;
server {
    listen 80;
    server_name api.myapp.com;
    location /api/ {
        proxy_pass http://backend_app;
    }
}
```

Redirect HTTPS

```
server {
    listen 80;
    server_name example.com;
    return 301 https://$host$request_uri;

server {
    listen 443;
    server_name example.com;

    ssl_certificate /etc/ssl/certs/example.crt;
    ssl_certificate_key /etc/ssl/private/example.key;
}
```

SAFE RELOAD WORKFLOW

```
# Check configuration syntax errrors
sudo  nginx -t

# Safe reload without downtime
sudo nginx -s reload

# Verify nginx status
sudo systemctl status nginx
```

```
sudo nginx -t && sudo nginx -s reload
```

BEST PRACTICES

- Global config only in **nginx.conf**
- Use include to modularize configuration
- Each website/service use its own file.
- Name file descriptively base on function.

ANTI-PATTERNS

- All config into 1 file.
- Using meaningless filenames like `test.conf`,`temp.conf`
- Duplication configuration blocks.

DEMONSTRATION

SETUP & VIDEO

Speaker: Hoàng Minh Giang

COMPARISONS & CONCLUSION

Speaker: Biện Xuân An

AN ALTERNATIVE

QUICK COMPARISON

	NGINX	Apache httpd
Model	Event-driven, asynchronous — one worker process handles thousands of connections	Process/thread-based (MPM prefork/worker/event) — traditionally one process or thread per connection
Config	Central config, edited by admin, reloaded	Supports **.htaccess** — per-directory overrides, no reload needed
Best known for	Reverse proxy, load balancer, static files, TLS termination	Flexibility, module ecosystem, shared hosting

WHEN TO USE
NGINX

High volumes of static content / concurrent connections

Reverse Proxy / Load Balancer

Lower memory usage

Deploying Containers / Kubernetes

WHEN TO USE APACHE

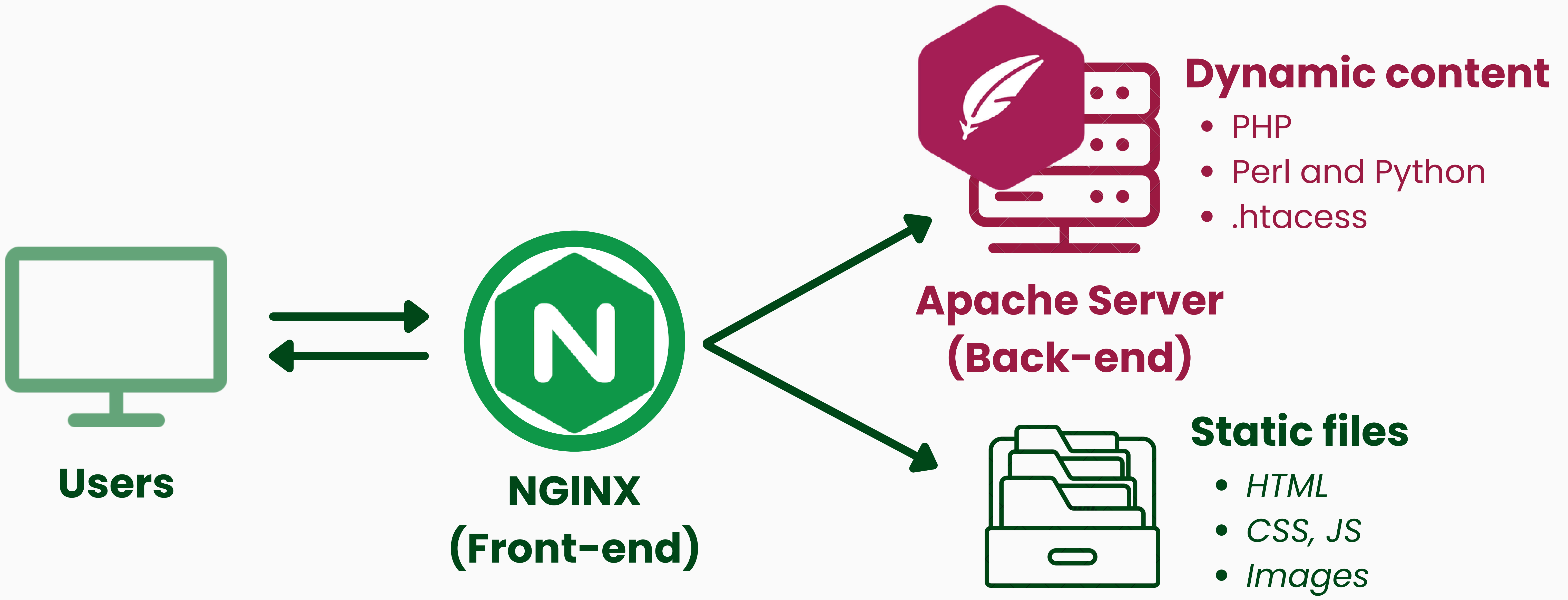
.htaccess flexibility / shared hosting

Apache module dependency

Complex mod_rewrite rule chains

Tech Stack

HYBRID SETUP: COMBINE NGINX AND APACHE



Nginx *acts as* ***front-end reverse proxy*** *with*
Apache *acts as* ***back-end application server***

COMMON MISTAKES

Mistake	Consequences	Fix
Not testing config before reloading	A syntax error can take the server down on reload/restart	Always run **sudo nginx -t** before **systemctl reload nginx**
Using a hostname instead of an IP in **listen** or **upstream** blocks	Hostname may fail to resolve at boot/restart - prevent NGINX from binding its socket	Using IP addresses where possible
Not raising file descriptor limits (**worker_rlimit_nofile**) alongside **worker_connections**	Workers can silently hit the ceiling under load, triggers "**Too many open files**" errors	Set **worker_rlimit_nofile** to at least 2× **worker_connections**

BEST PRACTICES

Never run NGINX as root

Mitigate damage in the event of a server attack (run with a dedicated, low-privilege user)

Constantly monitor error logs

Detect issues early before they escalate into major outages

Fine-tuning buffer sizes (client_body_buffer_size, proxy_buffer_size...)

An excessively small buffer causes high disk I/O, while an excessively large buffer leads to resource waste or crashes under high load

Monitor system resources (CPU, RAM, disk space)

Prevent NGINX from hanging or becoming unresponsive due to resource exhaustion

KEY TAKEAWAYS

NGINX acts as web server + reverse proxy + load balancer + TLS terminator, offloading work from the backend

Its event-driven, master–worker architecture handles massive concurrency with minimal resources

Not always "better" than Apache — the right tool depends on the workload. Most failures come from misconfiguration, not the tool itself